

CSCE 701 CYBERSECURITY · MODULE 07 · LECTURE NOTES

Secrets and Keys

The hard part of applied cryptography.

MODULE OUTCOMES

Explain why key management rather than cryptography is the failure point.

Design secret storage for an application.

Explain rotation and why it is the hard requirement.

Explain the hardware-backed options and what each buys.

Review a system for secret-handling failures.

1 Where it goes wrong

THE PRIMITIVES WORK; THE KEY MANAGEMENT DOES NOT

Modern authenticated encryption, digital signatures, and key agreement are, when used correctly, not the weak point of a system — CSCE 711 covers them, and the practical advice there is to use a high-level library and not to select primitives or modes yourself.

The failures are in the keys: committed to a source repository, written to a log, baked into a container image, shared between production and staging, never rotated in five years, or held in the memory of a service that had no need of them. Every one of those is a configuration or process failure rather than a mathematical one.

And they are mostly detectable. Secret scanning finds committed credentials reliably, and the major code-hosting platforms now scan by default and notify the issuing provider — which has usefully converted a silent failure into a noisy one, and is one of the more effective interventions of the last decade.

So this module is about plumbing. Which is where the incidents are — and a course that spent its cryptography time on cipher modes rather than on key handling would be addressing the wrong problem, which is why the division between this course and CSCE 711 falls where it does.

Where the secret lives	The problem	Verdict
In source code	It is in every clone, in the history, forever — and in every fork and every backup.	Never. And when found, rotate rather than delete (§4's footnote).
In a configuration file inside the container image	It is in every copy of the image in every registry, readable by anyone who can pull it.	Avoid — and note that image layers retain deleted files.

Where the secret lives	The problem	Verdict
In an environment variable	Visible in process listings, inherited by every child process, and captured in crash dumps and error reporters.	Acceptable and not ideal — the crash-dump path is the one that surprises people.
In a file mounted at runtime, on tmpfs	Readable by the process and by anything that can read its filesystem; not present in the image.	Good.
Fetches at runtime from a secret manager	Requires an identity to authenticate with — which is §2's bootstrap problem.	Better — and crucially, rotation becomes possible (§3).
Never leaves an HSM or KMS; operations are performed there	Every cryptographic operation becomes a network call, with latency and an availability dependency.	Best, for any key you can use remotely.

The last row is the real goal and is worth stating as an architectural principle: a key that never exists in your process's memory cannot be leaked by your process — not by a memory disclosure bug, not by a crash dump, not by a core file, and not by an attacker with code execution. **Which converts a secrecy problem into an authorisation and audit problem, and those are tractable with Module 03's machinery** while 'keep this value secret in a process an attacker may control' is not.

2 The bootstrap problem

EVERY SECRET MANAGEMENT SCHEME HAS A BOOTSTRAP PROBLEM

To fetch a secret from a secret manager you must authenticate to the secret manager — which requires a credential, which is itself a secret, which must be stored somewhere. The problem recurses.

The wrong answer is a long-lived token in a configuration file, because that has moved the problem rather than solved it — you now have one secret in a file instead of twenty, which is an improvement in blast radius and not a solution.

The right answer is a *platform-provided identity* — a cloud instance role, a Kubernetes service account token projected by the kubelet, a TPM-attested device identity, or a workload identity federated from the orchestrator — where the platform attests to who you are and no long-lived stored secret is involved at all.

So the bootstrap is resolved by moving it into infrastructure that can attest to identity, which is the one component positioned to do so. And if you have no such infrastructure, then the bootstrap secret is the one you protect hardest, scope most narrowly, and rotate most carefully — which is an honest answer rather than a good one.

3 Rotation

WHY YOU NEED IT

a leaked key must be replaceable without an outage
 a departing employee's access must actually end
 and a key in use for years accumulates exposure --
 through backups, logs, and former staff

WHY IT IS HARD: everything that holds or verifies the key
 must learn the new one, and they do not all learn it at the
 same moment. So there is necessarily a window during which
 BOTH keys are in use.

WHICH MEANS THE DESIGN MUST SUPPORT TWO VALID KEYS AT ONCE:

phase 1: verifiers accept OLD and NEW
 phase 2: signers and encryptors switch to NEW
 phase 3: OLD is withdrawn
 -- a three-phase rollout, and the system has to be
 built for it from the beginning

SO: KEY IDENTIFIERS EVERYWHERE. Every ciphertext, every
 token, and every signature carries an identifier saying
 which key produced it. Without that you cannot roll over at
 all; you can only cut over, with an outage and a window
 during which old artefacts cannot be read.

AND TEST THE ROTATION. An untested rotation procedure will
 be attempted for the first time during an incident, which
 is the worst possible moment -- so rotate on a schedule
 even when nothing is wrong, because that is the only way to
 know the procedure works.

Key identifiers in every artefact are the design decision that makes rotation possible, and retrofitting them produces an outage — because existing artefacts carry no identifier, so you must either try every key or accept that old data becomes unreadable. **It costs a few bytes per artefact and it is the difference between a routine rotation and an incident**, which makes it one of the highest-value early design decisions in this course.

4 Reviewing, and the hardware options

- **Scan the repository history, not merely the working tree. A secret that was committed and then deleted is still in the history, in every clone, and must be rotated** — and the tools for this are free.
- **Grep the logs for secret-shaped strings** — long base64 or hex values, known prefixes, authorisation headers. **Secrets in logs is a large and very quiet class**, because logs are copied to aggregators, retained for years, and readable by a wide audience.
- **Check whether any secret is shared between environments. A production key present in a staging system inherits staging's security posture**, which is invariably weaker — and staging is where the test data and the broad access are.

- **Ask when each secret was last rotated**, and more pointedly **whether the rotation procedure has ever been executed at all** (§3).
- **And ask who can read each secret.** 'Everyone in engineering' is the common answer and the wrong one (Module 03's least privilege). **Rotate rather than delete:** once a secret has been committed, logged, or shared, **it must be treated as compromised** — removing the text does not un-disclose it, and the cost of rotating is almost always less than the cost of being wrong.

THE HARDWARE-BACKED OPTIONS, AND WHAT EACH BUYS

A cloud key management service: keys never leave the service, and you call it to encrypt, decrypt, or sign — which turns key secrecy into an authorisation and audit problem (§1's architectural point), and both are tractable. **The default choice for most systems.**

A hardware security module: the same model, with a physical tamper boundary, a stronger attestation story, and a compliance certification — **used where a regulator requires it, or where the key is extremely valuable (a root signing key, a certificate authority).**

A TPM or a device secure element: binds a key to a specific physical device, so the key cannot be copied to another machine — **which is what makes device identity, measured boot, and full-disk encryption work** (Module 04 §3's device posture signal).

And a hardware security key: binds a key to a physical object a person holds — **which is Module 02 §1's phishing resistance**, and is **the same mechanism at a different scale:** the key cannot be exfiltrated because it cannot leave the device, and the signature is bound to the origin because the device enforces it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OWASP — Secrets Management Cheat Sheet (free)	The working reference for §1 and §4 , including the storage hierarchy and the scanning guidance.
Google — Building Secure and Reliable Systems, the key management material (free PDF)	§2's bootstrap problem and §3's rotation , from people who operate it — and the identity-rather-than-secret framing is theirs.
Meli, McNiece & Reaves — How Bad Can It Get? (free)	§1's committed-secret problem, measured — the scale is larger than you would guess.
NIST SP 800-57 — Recommendation for Key Management (free)	§3's rotation and lifetime guidance , and the key-state model is worth borrowing even if the document is long.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Run a secret scanner over your repository history** and report what it finds.
- 2 **For anything found, write the rotation plan** rather than deleting the line.
- 3 **Grep a month of your logs** for secret-shaped strings.
- 4 **Move one secret from an environment variable to a mounted file**, and then to a secret manager.
- 5 **Identify your bootstrap credential** and say whether it is a stored secret or a platform identity.
- 6 **Add key identifiers** to a system that lacks them, and report what it took.
- 7 **Execute a key rotation** on a non-critical key, and time it.
- 8 **Write the rotation runbook** and have someone else follow it.
- 9 **Move one signing operation into a KMS** so the key never enters your process.
- 10 **Run §4's five-point review** on a system you operate.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is key management rather than cryptography the failure point?
- 2 Give the six storage locations, worst to best, and the problem with each.
- 3 Why is 'never leaves the HSM' the architectural goal, and what does it convert the problem into?
- 4 State the bootstrap problem and the right answer.
- 5 Why is rotation hard, and what three phases does it require?
- 6 What design decision makes rotation possible?
- 7 Why rotate on a schedule when nothing is wrong?
- 8 Give five review questions and the rotate-rather-than-delete rule.
- 9 Name four hardware-backed options and what each binds a key to.